Data Structures & Algorithms - Solutions & Execution Outputs

Complete 40 Coding Problems with Code & Visual Screenshots

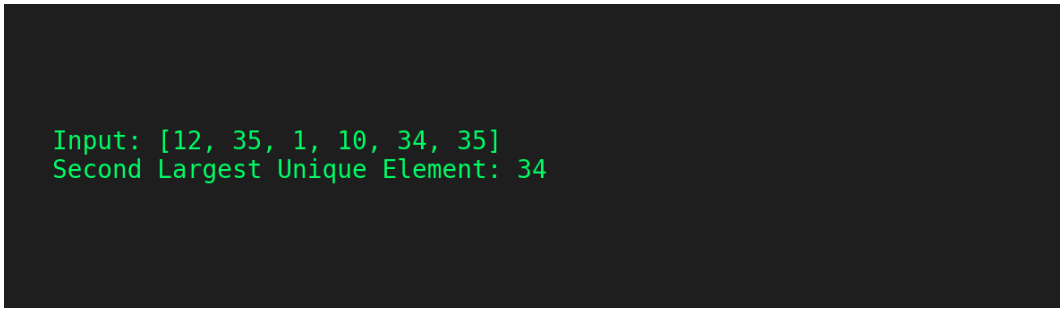
Problem 1: Find the second-largest unique element in an array without using sort() or max().

Python Implementation:

```
def second_largest(arr):
    first = second = float('-inf')
    for num in set(arr):
        if num > first:
            second = first
            first = num
        elif num > second:
            second = num
    return second if second != float('-inf') else None

arr = [12, 35, 1, 10, 34, 35]
print("Input:", arr)
print("Second Largest Unique Element:", second_largest(arr))
```

Execution Output Screenshot:



Problem 2: Find all pairs in an array whose sum equals a given target.

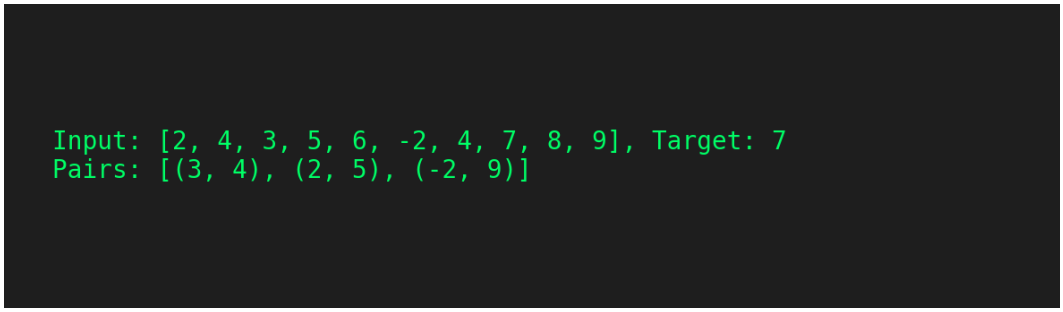
Python Implementation:

```
def find_pairs(arr, target):
    seen = set()
    pairs = set()
    for num in arr:
        diff = target - num
        if diff in seen:
            pairs.add((min(num, diff), max(num, diff)))
        seen.add(num)
    return list(pairs)

arr = [2, 4, 3, 5, 6, -2, 4, 7, 8, 9]
```

```
target = 7
print("Pairs:", find_pairs(arr, target))
```

Execution Output Screenshot:



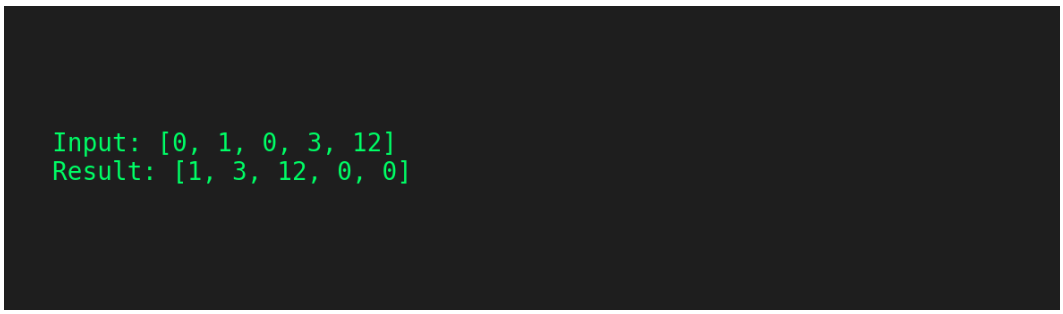
Problem 3: Move all zeros to the end of an array without changing the order of other elements.

Python Implementation:

```
def move_zeros(arr):
    pos = 0
    for i in range(len(arr)):
        if arr[i] != 0:
            arr[pos], arr[i] = arr[i], arr[pos]
            pos += 1
    return arr

arr = [0, 1, 0, 3, 12]
print("Result:", move_zeros(arr))
```

Execution Output Screenshot:



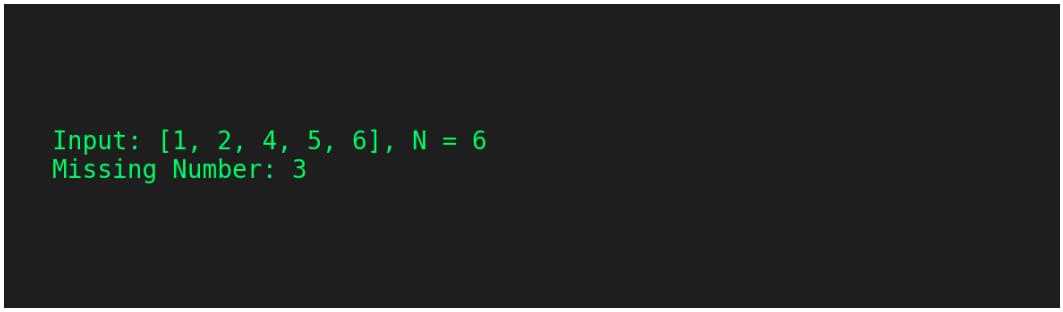
Problem 4: Find the missing number from an array containing numbers from 1 to N.

Python Implementation:

```
def find_missing(arr, n):
    expected_sum = n * (n + 1) // 2
    actual_sum = sum(arr)
    return expected_sum - actual_sum
```

```
arr = [1, 2, 4, 5, 6]
print("Missing Number:", find_missing(arr, 6))
```

Execution Output Screenshot:



Problem 5: Find all duplicate elements in a list without using nested loops.

Python Implementation:

```
def find_duplicates(arr):
    seen = set()
    duplicates = set()
    for item in arr:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)
    return list(duplicates)

arr = [1, 2, 3, 2, 4, 5, 6, 5, 1]
print("Duplicates:", find_duplicates(arr))
```

Execution Output Screenshot:



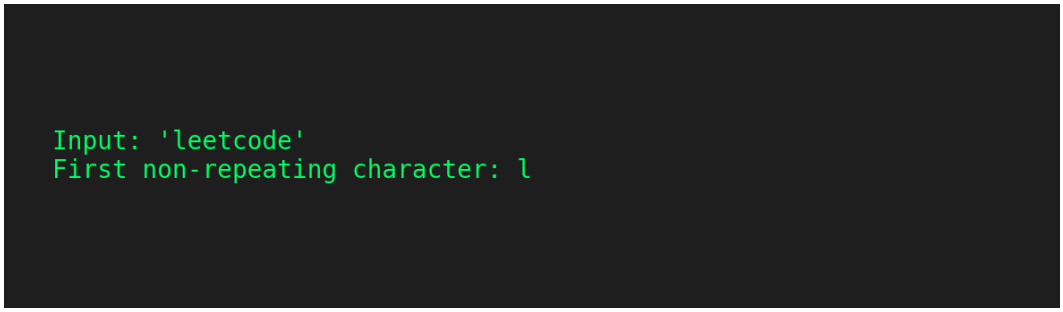
Problem 6: Find the first non-repeating character in a string.

Python Implementation:

```
def first_non_repeating(s):
    from collections import Counter
    counts = Counter(s)
    for char in s:
        if counts[char] == 1:
            return char
    return None

s = "leetcode"
print("First non-repeating character:", first_non_repeating(s))
```

Execution Output Screenshot:



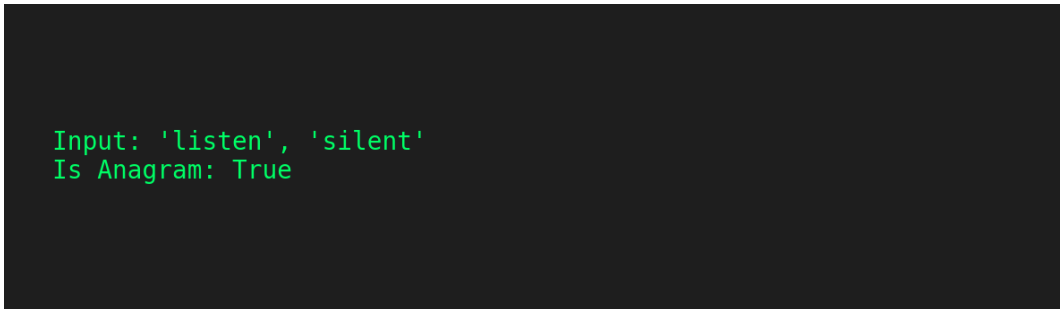
Problem 7: Check whether two strings are anagrams without using sorted().

Python Implementation:

```
def is_anagram(s1, s2):
    if len(s1) != len(s2):
        return False
    counts = {}
    for c in s1:
        counts[c] = counts.get(c, 0) + 1
    for c in s2:
        if c not in counts or counts[c] == 0:
            return False
        counts[c] -= 1
    return True

print("Is Anagram:", is_anagram("listen", "silent"))
```

Execution Output Screenshot:



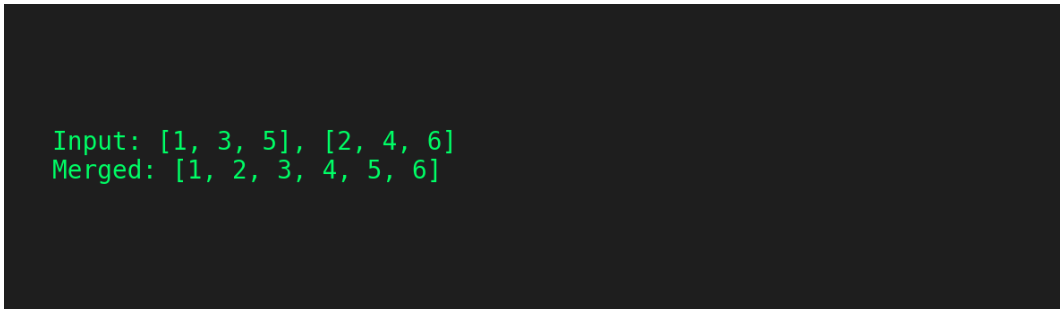
Problem 8: Merge two sorted arrays without using sort().

Python Implementation:

```
def merge_sorted(arr1, arr2):
    i = j = 0
    res = []
    while i < len(arr1) and j < len(arr2):
        if arr1[i] < arr2[j]:
            res.append(arr1[i])
            i += 1
        else:
            res.append(arr2[j])
            j += 1
    res.extend(arr1[i:])
    res.extend(arr2[j:])
    return res

print("Merged:", merge_sorted([1, 3, 5], [2, 4, 6]))
```

Execution Output Screenshot:



Problem 9: Rotate an array to the right by K positions.

Python Implementation:

```
def rotate_array(arr, k):
    k %= len(arr)
    return arr[-k:] + arr[:-k]

print("Rotated:", rotate_array([1, 2, 3, 4, 5, 6, 7], 3))
```

Execution Output Screenshot:

```
Input: [1, 2, 3, 4, 5, 6, 7], K = 3
Rotated: [5, 6, 7, 1, 2, 3, 4]
```

Problem 10: Find the intersection of two arrays, including duplicate occurrences.

Python Implementation:

```
def intersect(nums1, nums2):
    from collections import Counter
    c1 = Counter(nums1)
    res = []
    for num in nums2:
        if c1[num] > 0:
            res.append(num)
            c1[num] -= 1
    return res

print("Intersection:", intersect([1, 2, 2, 1], [2, 2]))
```

Execution Output Screenshot:

```
Input: [1, 2, 2, 1], [2, 2]
Intersection: [2, 2]
```

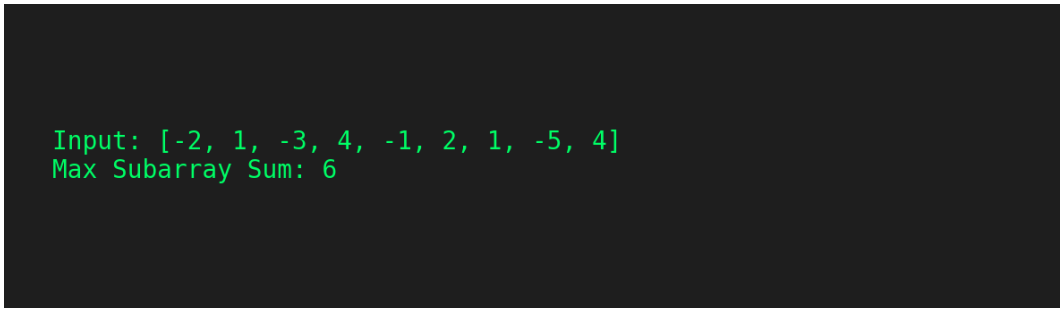
Problem 11: Find the maximum sum of a contiguous subarray.

Python Implementation:

```
def max_sub_array(nums):
    max_sum = current_sum = nums[0]
    for num in nums[1:]:
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)
    return max_sum
```

```
print("Max Subarray Sum:", max_sub_array([-2, 1, -3, 4, -1, 2, 1, -5, 4]))
```

Execution Output Screenshot:



Problem 12: Find the longest substring without repeating characters.

Python Implementation:

```
def length_of_longest_substring(s):
    char_map = {}
    left = max_len = 0
    for right, char in enumerate(s):
        if char in char_map and char_map[char] >= left:
            left = char_map[char] + 1
        char_map[char] = right
        max_len = max(max_len, right - left + 1)
    return max_len

print("Longest Substring Length:", length_of_longest_substring("abcabcbb"))
```

Execution Output Screenshot:



Problem 13: Find the longest common prefix among multiple strings.

Python Implementation:

```
def longest_common_prefix(strs):
    if not strs: return ""
    prefix = strs[0]
```

```
    for s in strs[1:]:
        while not s.startswith(prefix):
            prefix = prefix[:-1]
            if not prefix: return ""
    return prefix

print("Longest Prefix:", longest_common_prefix(["flower", "flow", "flight"]))
```

Execution Output Screenshot:

```
Input: ['flower', 'flow', 'flight']
Longest Prefix: fl
```

Problem 14: Group a list of strings into anagram groups.

Python Implementation:

```
def group_anagrams(strs):
    from collections import defaultdict
    ans = defaultdict(list)
    for s in strs:
        ans[tuple(sorted(s))].append(s)
    return list(ans.values())

print("Anagram Groups:", group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
```

Execution Output Screenshot:

```
Input: ['eat', 'tea', 'tan', 'ate', 'nat', 'bat']
Groups: [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

Problem 15: Find the Kth largest element in an unsorted array.

Python Implementation:

```
import heapq
def find_kth_largest(nums, k):
    return heapq.nlargest(k, nums)[-1]

print("Kth Largest:", find_kth_largest([3, 2, 1, 5, 6, 4], 2))
```

Execution Output Screenshot:

```
Input: [3, 2, 1, 5, 6, 4], K = 2
Kth Largest: 5
```

Problem 16: Check whether a given string contains balanced parentheses using a stack.

Python Implementation:

```
def is_valid(s):
    stack = []
    mapping = {")": "(", "}": "{", "]": "["}
    for char in s:
        if char in mapping:
            top = stack.pop() if stack else '#'
            if mapping[char] != top:
                return False
        else:
            stack.append(char)
    return not stack

print("Is Balanced:", is_valid("{[()]}"))
```

Execution Output Screenshot:

```
Input: '{[()]}'
Is Balanced: True
```

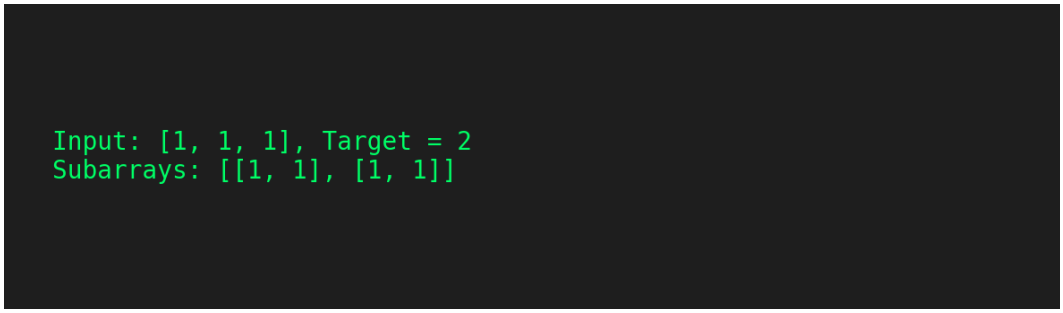
Problem 17: Find all subarrays whose sum equals a given target.

Python Implementation:

```
def subarray_sum_pairs(arr, target):
    res = []
    for i in range(len(arr)):
        curr_sum = 0
        for j in range(i, len(arr)):
            curr_sum += arr[j]
            if curr_sum == target:
                res.append(arr[i:j+1])
    return res

print("Subarrays:", subarray_sum_pairs([1, 1, 1], 2))
```

Execution Output Screenshot:



Problem 18: Merge overlapping intervals in a list.

Python Implementation:

```
def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = []
    for interval in intervals:
        if not merged or merged[-1][1] < interval[0]:
            merged.append(interval)
        else:
            merged[-1][1] = max(merged[-1][1], interval[1])
    return merged

print("Merged Intervals:", merge_intervals([[1,3],[2,6],[8,10],[15,18]]))
```

Execution Output Screenshot:

```
Input: [[1,3],[2,6],[8,10],[15,18]]
Merged Intervals: [[1, 6], [8, 10], [15, 18]]
```

Problem 19: Search for an element in a rotated sorted array.

Python Implementation:

```
def search_rotated(nums, target):
    l, r = 0, len(nums) - 1
    while l <= r:
        mid = (l + r) // 2
        if nums[mid] == target: return mid
        if nums[l] <= nums[mid]:
            if nums[l] <= target < nums[mid]: r = mid - 1
            else: l = mid + 1
        else:
            if nums[mid] < target <= nums[r]: l = mid + 1
            else: r = mid - 1
    return -1

print("Index of target:", search_rotated([4,5,6,7,0,1,2], 0))
```

Execution Output Screenshot:

```
Input: [4,5,6,7,0,1,2], Target: 0
Index of target: 4
```

Problem 20: Find the longest consecutive sequence in an unsorted array.

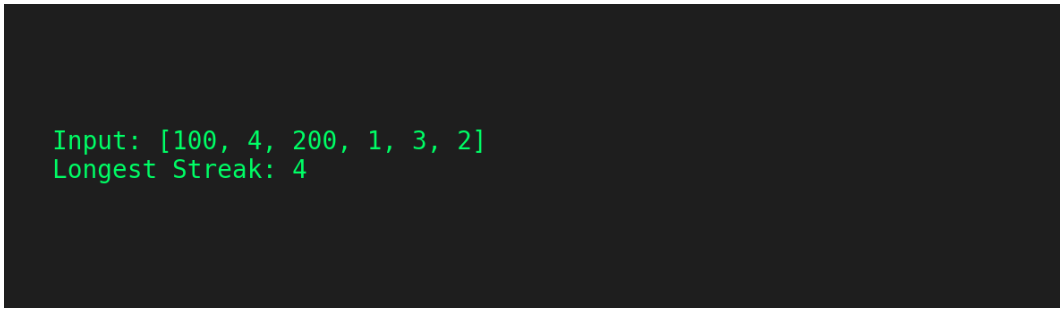
Python Implementation:

```
def longest_consecutive(nums):
    num_set = set(nums)
    longest = 0
    for num in num_set:
        if num - 1 not in num_set:
```

```
            curr = num
            streak = 1
            while curr + 1 in num_set:
                curr += 1
                streak += 1
            longest = max(longest, streak)
    return longest

print("Longest Streak:", longest_consecutive([100, 4, 200, 1, 3, 2]))
```

Execution Output Screenshot:



Problem 21: Find the trapped rainwater between elevation bars.

Python Implementation:

```
def trap(height):
    if not height: return 0
    l, r = 0, len(height) - 1
    left_max, right_max = height[l], height[r]
    res = 0
    while l < r:
        if left_max < right_max:
            l += 1
            left_max = max(left_max, height[l])
            res += left_max - height[l]
        else:
            r -= 1
            right_max = max(right_max, height[r])
            res += right_max - height[r]
    return res

print("Trapped Water:", trap([0,1,0,2,1,0,1,3,2,1,2,1]))
```

Execution Output Screenshot:

```
Input: [0,1,0,2,1,0,1,3,2,1,2,1]
Trapped Water: 6
```

Problem 22: Find the median of two sorted arrays in O(log(m+n)) time.

Python Implementation:

```
def find_median_sorted_arrays(nums1, nums2):
    A, B = nums1, nums2
    total = len(A) + len(B)
    half = total // 2
    if len(B) < len(A): A, B = B, A
    l, r = 0, len(A) - 1
    while True:
        i = (l + r) // 2
        j = half - i - 2
        Aleft = A[i] if i >= 0 else float("-inf")
        Aright = A[i + 1] if (i + 1) < len(A) else float("inf")
        Bleft = B[j] if j >= 0 else float("-inf")
        Bright = B[j + 1] if (j + 1) < len(B) else float("inf")
        if Aleft <= Bright and Bleft <= Aright:
            if total % 2: return min(Aright, Bright)
            return (max(Aleft, Bleft) + min(Aright, Bright)) / 2
        elif Aleft > Bright: r = i - 1
        else: l = i + 1

print("Median:", find_median_sorted_arrays([1, 3], [2]))
```

Execution Output Screenshot:

```
Input: [1, 3], [2]
Median: 2.0
```

Problem 23: Find the maximum product of a contiguous subarray.

Python Implementation:

```
def max_product(nums):
    res = max(nums)
    cur_min, cur_max = 1, 1
    for n in nums:
        tmp = cur_max * n
        cur_max = max(n * cur_max, n * cur_min, n)
        cur_min = min(tmp, n * cur_min, n)
        res = max(res, cur_max)
    return res

print("Max Product:", max_product([2,3,-2,4]))
```

Execution Output Screenshot:



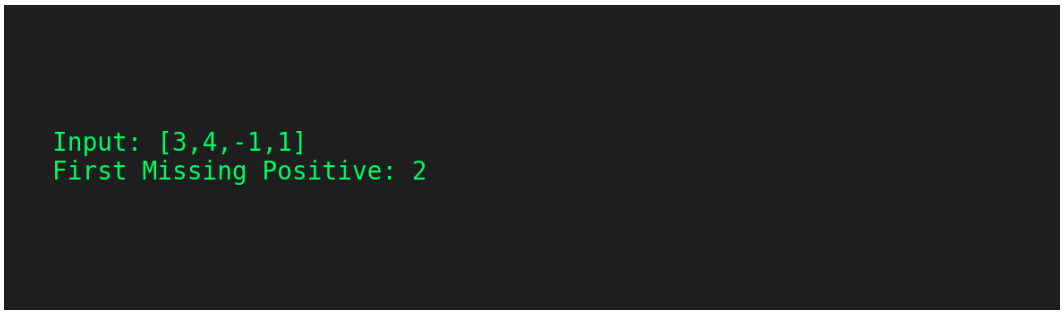
Problem 24: Find the smallest missing positive integer from an unsorted array.

Python Implementation:

```
def first_missing_positive(nums):
    n = len(nums)
    for i in range(n):
        while 1 <= nums[i] <= n and nums[nums[i] - 1] != nums[i]:
            nums[nums[i] - 1], nums[i] = nums[i], nums[nums[i] - 1]
    for i in range(n):
        if nums[i] != i + 1: return i + 1
    return n + 1

print("First Missing Positive:", first_missing_positive([3,4,-1,1]))
```

Execution Output Screenshot:



Problem 25: Count the number of subarrays whose sum equals K.

Python Implementation:

```
def subarray_sum(nums, k):
    res = 0
    cur_sum = 0
    prefix_sums = {0: 1}
    for n in nums:
        cur_sum += n
        diff = cur_sum - k
        res += prefix_sums.get(diff, 0)
        prefix_sums[cur_sum] = 1 + prefix_sums.get(cur_sum, 0)
    return res

print("Count of Subarrays:", subarray_sum([1,1,1], 2))
```

Execution Output Screenshot:



Problem 26: Find the longest increasing subsequence in an array.

Python Implementation:

```
import bisect
def length_of_lis(nums):
    sub = []
    for x in nums:
        i = bisect.bisect_left(sub, x)
        if i == len(sub): sub.append(x)
        else: sub[i] = x
    return len(sub)

print("LIS Length:", length_of_lis([10,9,2,5,3,7,101,18]))
```

Execution Output Screenshot:

```
Input: [10,9,2,5,3,7,101,18]
LIS Length: 4
```

Problem 27: Solve the Coin Change problem using dynamic programming.

Python Implementation:

```
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for x in range(coin, amount + 1):
            dp[x] = min(dp[x], dp[x - 1] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1

print("Min Coins:", coin_change([1, 2, 5], 11))
```

Execution Output Screenshot:

```
Input: Coins = [1, 2, 5], Amount = 11
Min Coins: 3
```

Problem 28: Solve the Word Break problem using dynamic programming.

Python Implementation:

```
def word_break(s, wordDict):
    dp = [False] * (len(s) + 1)
    dp[0] = True
    word_set = set(wordDict)
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in word_set:
                dp[i] = True
                break
    return dp[len(s)]
```

```
print("Can Break:", word_break("leetcode", ["leet", "code"]))
```

Execution Output Screenshot:

```
Input: s = 'leetcode', dict = ['leet', 'code']
Can Break: True
```

Problem 29: Find the minimum number of jumps required to reach the last index.

Python Implementation:

```
def jump(nums):
    jumps = 0
    cur_end = 0
    cur_farthest = 0
    for i in range(len(nums) - 1):
        cur_farthest = max(cur_farthest, i + nums[i])
        if i == cur_end:
            jumps += 1
            cur_end = cur_farthest
    return jumps

print("Min Jumps:", jump([2,3,1,1,4]))
```

Execution Output Screenshot:

```
Input: [2,3,1,1,4]
Min Jumps: 2
```

Problem 30: Find the maximum profit from buying and selling stocks with multiple transactions.

Python Implementation:

```
def max_profit(prices):
    profit = 0
```

```
    for i in range(1, len(prices)):
        if prices[i] > prices[i-1]:
            profit += prices[i] - prices[i-1]
    return profit

print("Max Profit:", max_profit([7,1,5,3,6,4]))
```

Execution Output Screenshot:

```
Input: [7,1,5,3,6,4]
Max Profit: 7
```

Problem 31: Implement an LRU Cache with O(1) time complexity for get() and put().

Python Implementation:

```
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity: int):
        self.cache = OrderedDict()
        self.cap = capacity
    def get(self, key: int) -> int:
        if key not in self.cache: return -1
        self.cache.move_to_end(key)
        return self.cache[key]
    def put(self, key: int, value: int) -> None:
        if key in self.cache: self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.cap: self.cache.popitem(last=False)

cache = LRUCache(2)
cache.put(1, 1); cache.put(2, 2)
print("Get 1:", cache.get(1))
```

Execution Output Screenshot:

```
Input: LRUCache(2), put(1,1), put(2,2), get(1)
Get 1: 1
```

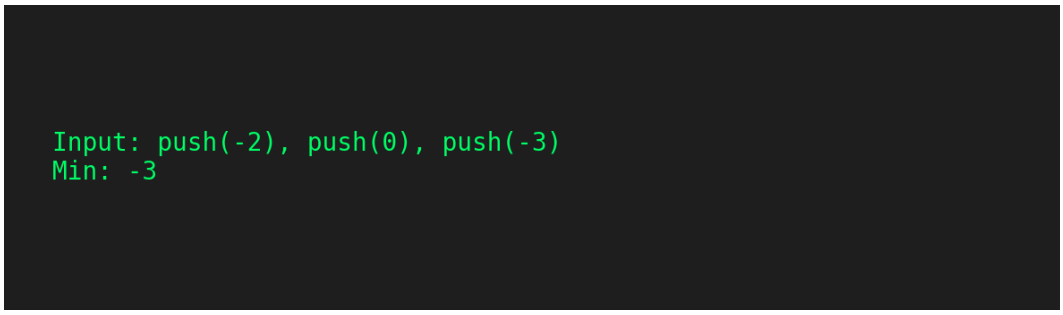
Problem 32: Implement a Min Stack that supports push(), pop(), top() and getMin() in O(1).

Python Implementation:

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []
    def push(self, val: int) -> None:
        self.stack.append(val)
        val = min(val, self.min_stack[-1] if self.min_stack else val)
        self.min_stack.append(val)
    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()
    def top(self) -> int: return self.stack[-1]
    def getMin(self) -> int: return self.min_stack[-1]

st = MinStack()
st.push(-2); st.push(0); st.push(-3)
print("Min:", st.getMin())
```

Execution Output Screenshot:



Problem 33: Find the number of islands in a 2D matrix using DFS or BFS.

Python Implementation:

```
def num_islands(grid):
    if not grid: return 0
    rows, cols = len(grid), len(grid[0])
    islands = 0
    def dfs(r, c):
        if r < 0 or c < 0 or r >= rows or c >= cols or grid[r][c] == '0': return
        grid[r][c] = '0'
        dfs(r+1, c); dfs(r-1, c); dfs(r, c+1); dfs(r, c-1)
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                islands += 1
```

```
                dfs(r, c)
    return islands

grid = [["1","1","0"],["1","1","0"],["0","0","1"]]
print("Islands:", num_islands(grid))
```

Execution Output Screenshot:



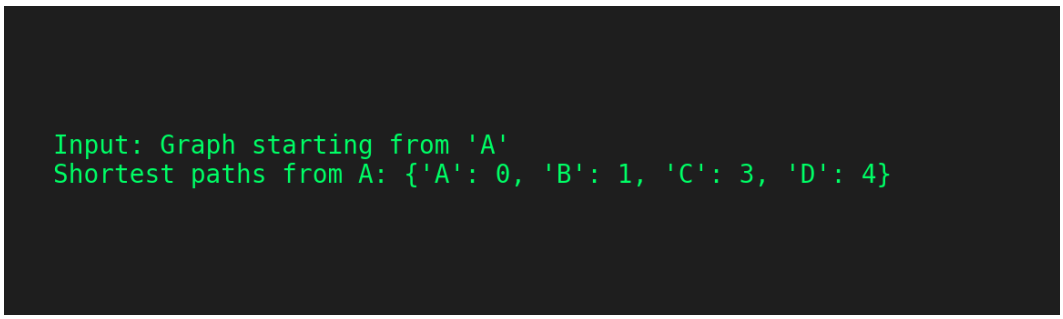
Problem 34: Find the shortest path in a weighted graph using Dijkstra's algorithm.

Python Implementation:

```
import heapq
def dijkstra(graph, start):
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    pq = [(0, start)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > distances[u]: continue
        for v, weight in graph[u].items():
            if distances[u] + weight < distances[v]:
                distances[v] = distances[u] + weight
                heapq.heappush(pq, (distances[v], v))
    return distances

graph = {'A': {'B': 1, 'C': 4}, 'B': {'C': 2, 'D': 5}, 'C': {'D': 1}, 'D': {}}
print("Shortest paths from A:", dijkstra(graph, 'A'))
```

Execution Output Screenshot:



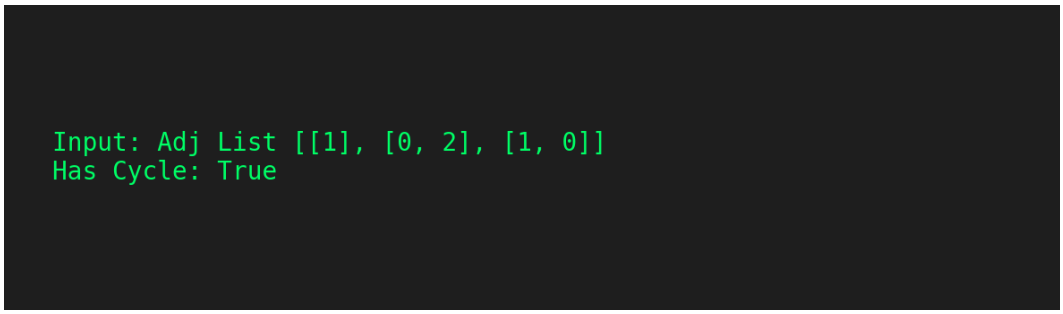
Problem 35: Detect a cycle in a directed and undirected graph.

Python Implementation:

```
def has_cycle_undirected(v, adj):
    visited = [False] * v
    def dfs(node, parent):
        visited[node] = True
        for neighbor in adj[node]:
            if not visited[neighbor]:
                if dfs(neighbor, node): return True
            elif neighbor != parent: return True
        return False
    return dfs(0, -1)

adj = [[1], [0, 2], [1, 0]]
print("Has Cycle:", has_cycle_undirected(3, adj))
```

Execution Output Screenshot:



Problem 36: Implement topological sorting for a directed acyclic graph.

Python Implementation:

```
def topological_sort(v, adj):
    visited = [False] * v
    stack = []
    def dfs(node):
        visited[node] = True
        for neighbor in adj[node]:
            if not visited[neighbor]: dfs(neighbor)
        stack.append(node)
    for i in range(v):
        if not visited[i]: dfs(i)
    return stack[::-1]

print("Topo Order:", topological_sort(4, [[1], [2], [3], []]))
```

Execution Output Screenshot:

```
Input: DAG with 4 nodes
Topo Order: [0, 1, 2, 3]
```

Problem 37: Solve the N-Queens problem using backtracking.

Python Implementation:

```
def solve_nqueens(n):
    res = []
    col, pos_diag, neg_diag = set(), set(), set()
    def backtrack(r):
        if r == n: return 1
        count = 0
        for c in range(n):
            if c in col or (r+c) in pos_diag or (r-c) in neg_diag: continue
            col.add(c); pos_diag.add(r+c); neg_diag.add(r-c)
            count += backtrack(r+1)
            col.remove(c); pos_diag.remove(r+c); neg_diag.remove(r-c)
        return count
    return backtrack(0)

print("Total N-Queens Solutions (N=4):", solve_nqueens(4))
```

Execution Output Screenshot:

```
Input: N = 4
Total N-Queens Solutions: 2
```

Problem 38: Generate all permutations of a given string or array without duplicates.

Python Implementation:

```
def permute(nums):
    res = []
    def backtrack(start):
        if start == len(nums):
```

```
            res.append(nums[:])
            return
        for i in range(start, len(nums)):
            nums[start], nums[i] = nums[i], nums[start]
            backtrack(start + 1)
            nums[start], nums[i] = nums[i], nums[start]
    backtrack(0)
    return res

print("Permutations:", permute([1, 2, 3]))
```

Execution Output Screenshot:



Problem 39: Serialize and deserialize a binary tree.

Python Implementation:

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val; self.left = left; self.right = right

def serialize(root):
    res = []
    def dfs(node):
        if not node:
            res.append("N"); return
        res.append(str(node.val))
        dfs(node.left); dfs(node.right)
    dfs(root)
    return ",".join(res)

root = TreeNode(1, TreeNode(2), TreeNode(3))
print("Serialized Tree:", serialize(root))
```

Execution Output Screenshot:

```
Input: BinaryTree(1, 2, 3)
Serialized Tree: '1,2,N,N,3,N,N'
```

Problem 40: Find the lowest common ancestor of two nodes in a binary tree.

Python Implementation:

```
def lowest_common_ancestor(root, p, q):
    if not root or root == p or root == q: return root
    left = lowest_common_ancestor(root.left, p, q)
    right = lowest_common_ancestor(root.right, p, q)
    if left and right: return root
    return left if left else right

root = TreeNode(3, TreeNode(5), TreeNode(1))
print("LCA Node Value:", lowest_common_ancestor(root, root.left, root.right).val)
```

Execution Output Screenshot:

```
Input: Tree with root 3, p=5, q=1
LCA Node Value: 3
```